

System Protection Mechanisms: SECCOMP, SELinux, and chroot

SECCOMP

Seccomp (Secure Computing Mode) is a Linux kernel feature that restricts the system calls a process can make. It lets a process load a Berkeley Packet Filter (BPF) policy to allow or deny each syscall before execution. In practice, seccomp is widely used in container runtimes: for example, Docker and Kubernetes use seccomp profiles to block unexpected syscalls. By filtering at the syscall level, seccomp drastically reduces the kernel's exposed attack surface. (It is not a complete sandbox by itself, so it is usually combined with other controls.)

SELinux

SELinux (Security-Enhanced Linux) is a kernel-integrated mandatory access control (MAC) system. Every process, file, and resource is labeled, and a policy specifies which labels may interact. By default, no access is allowed unless explicitly permitted by the policy. This means administrators can confine each process to a strict domain of actions; even if a service is compromised, SELinux limits what it can do. Originally developed by the NSA, SELinux has been integrated into mainstream Linux (it's enabled by default on RHEL) and is used on high-security servers and embedded systems like Android.

chroot

A chroot ("change root") operation changes a process's view of the filesystem root to a specified directory. After executing chroot, the process and its children see the given directory as "/" and cannot access files outside it. This effectively confines applications to a designated subtree (a "chroot jail"), which can be useful for testing, recovery, or isolating untrusted programs. However, chroot only limits filesystem access. It does *not* restrict system calls, networking, or other kernel interactions. In fact, processes with sufficient privilege can often break out of a chroot. Thus, chroot provides only a basic sandbox – additional security measures are needed for full isolation.

Comparative Analysis

- **Security Effectiveness:** SELinux provides the strongest confinement by enforcing system-wide MAC policies on all objects. Seccomp offers targeted protection by filtering syscalls, which greatly shrinks the attack surface, but it can only block exploits that rely on forbidden syscalls. Chroot offers the weakest protection: it simply changes the filesystem namespace and is easily bypassed if the process gains privileges.
- **Ease of Use:** Chroot is the simplest (one command creates a jail), but managing a functional environment inside it (copying binaries/libraries) can be tedious. Seccomp is moderately easy to use with prebuilt profiles (e.g. Docker's default), but creating custom BPF filters requires care. SELinux is the most complex: writing or adjusting policies and labels has a steep learning curve, even though many systems ship with default policies.
- **Flexibility:** SELinux is very flexible – virtually any resource or action can be labeled and controlled by policy. Seccomp is limited to syscall filtering only: it cannot restrict file or network access beyond what syscalls allow. Chroot is the least flexible: it only isolates the filesystem tree. It does not confine processes in any other dimension.
- **Performance Impact:** Chroot has negligible runtime overhead (only the initial namespace change). Seccomp adds a small cost for each syscall as the filter is evaluated (typically a few dozen nanoseconds per call), which is minimal for most workloads. SELinux adds overhead for every access check against the policy, but studies show this impact is small for normal use. In modern kernels with policy caching, typical applications see only a minor slowdown under SELinux.
- **Complexity:** SELinux is the most complex to configure and maintain (due to extensive policy and labeling). Seccomp is simpler: basic profiles can be used out-of-the-box, but deep customization requires understanding BPF. Chroot is simple to set up conceptually, though creating a usable chroot environment (with all needed files) can involve some work.

Use Cases and Deployment

- **SECCOMP:** Best suited for containerized and sandboxed applications. Container platforms (Docker, containerd, Kubernetes) use seccomp by default to block dangerous syscalls. Seccomp is also useful on regular Linux servers for sandboxing service binaries (e.g. letting a process drop unneeded syscalls). Embedded Linux systems with minimal syscall requirements can use seccomp profiles as a lightweight hardening measure.
- **SELinux:** Common on general-purpose Linux servers and distributions. For example, RHEL/CentOS/Fedora enable SELinux by default to confine daemons and system processes. Android (embedded/mobile) enforces SELinux to protect apps and system services. SELinux (or similar LSMs) is used in virtualization (e.g. sVirt for KVM/libvirt) and security-sensitive installations. It is recommended for any environment that requires strong, fine-grained isolation (e.g. databases, network services, government/military systems).
- **chroot:** Used for legacy or limited isolation tasks. Common use cases include creating a temporary chroot environment for system maintenance or recovery (e.g. from a rescue CD), building software in a clean environment, or setting up simple jailed services (such as an FTP/SFTP server). Chroot jails are seen on older Unix servers and in maintenance scripts, but in new deployments they have largely been superseded by containers or full VMs for stronger isolation.

Conclusion

Each of these mechanisms has a place in modern system security. SECCOMP and SELinux are essential for layered defense in Linux, especially in containers and multi-tenant servers. chroot, while limited, still has value in specialized or transitional contexts.